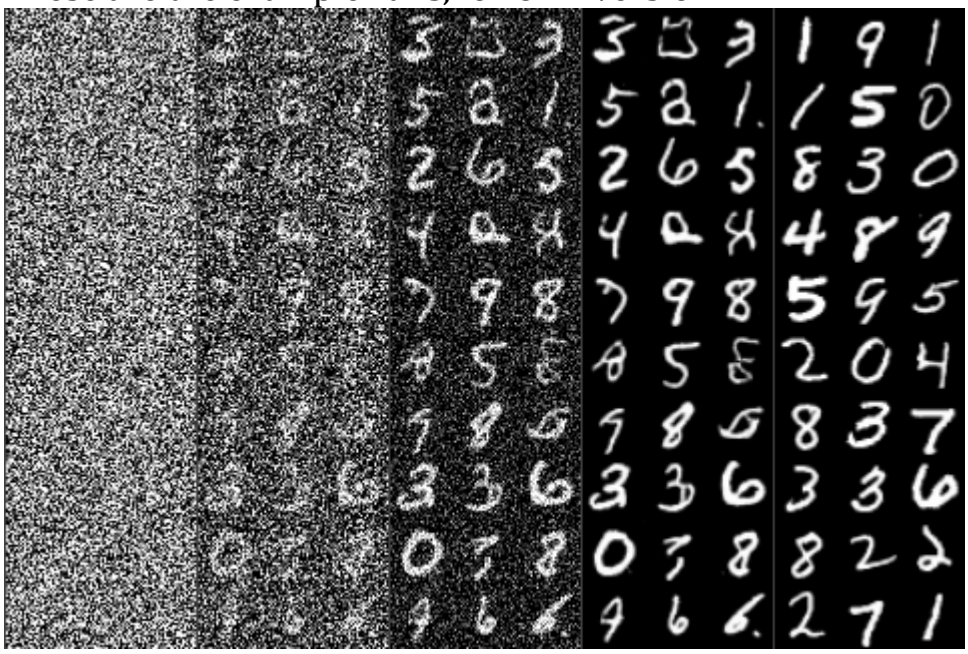
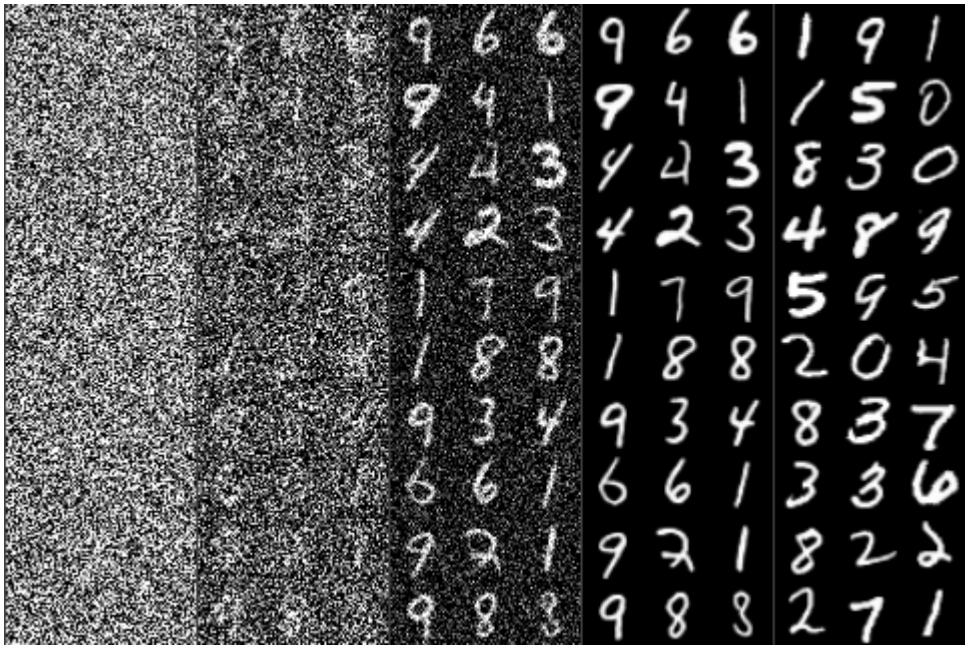
Practical Part - Question 1

Tuesday, 16 June 2026 16:48

These are the example runs, for ODE version:



And for the SDE version:



We also changed ex4.py to use noise=1.0 and a clean conditioning image, meaning the denoising starts from $\sigma_{max} = 80$ is essentially pure Gaussian noise. We did that to fairly compare the two samplers, since the SDE sampler needs enough noise levels to mix.

Algorithm: Stochastic VE Score Matching Sampler (SDE)

Input: x_0 (clean image), noise level p ∈ [0,1], trained denoiser D_θ

Output: trajectory of denoised samples $x_{\{T\}}, x_{\{T-1\}}, \ldots, x_0$

1. Compute $\sigma_{max} = schedule(p),\ \sigma_{min} = schedule(0)$
2. Build ladder: $[\sigma_T, \sigma_{\{T-1\}}, \ldots, \sigma_1, \sigma_0] = schedule(linspace(p, 0, T+1))$
3. Initialize: $x \leftarrow x_0 + \sigma_T \cdot z,\ \ z \sim N(0, I)$
4. For each step $i = T, T-1, \ldots, 1$:
   a. Predict clean image:
      $\hat{x}_0 = D_\theta(x, \sigma_i)$ // network denoises x at current noise level
   b. Compute score (relationship between denoiser and score in VE):
      $\nabla_x \log p(x) = \frac{(\hat{x}_0 - x)}{\sigma_i^2}$
   c. If $\sigma_{\{i-1\}} > 0$ (stochastic step):
      z ~ N(0, I) // fresh, independent noise — key difference from ODE
      $x \leftarrow \hat{x}_0 + \sigma_{\{i-1\}} \cdot z$ // re-noise the clean estimate at the next level
   Else (final step, $\sigma_{\{i-1\}} = 0$):
      x ← x̂_0 // return clean estimate directly
5. Return x

The key difference from the ODE version is step 4C: the ODE reuses $pred_{eps} = \frac{(x - \hat{x}_0)}{\sigma_i}$ (the same noise direction) and scales it to $\sigma_{\{i-1\}}$, effectively tracking one trajectory.
The SDE discards that direction and draws a fresh z at every step, like we learned
of langevin dynamics.

In practice, this required two small changes to the code- a stochastic flag was added to the sample() method in score_matching.py, and the Euler step inside the loop was replaced with $x = pred\_x\_0 + sigma\_end * randn$ when the flag is set. All other logic remained unchanged.

Practical Part- Question 2

Tuesday, 16 June 2026 16:53

To compare the two samplers against the real data distribution, we computed four moment-based test metrics over 256 images from each source. We additionally sampled 2000 images (in batches) for FID calculation. The metrics we chose were:

1. Pixel Mean - checks if overall brightness of the images matches the real distribution.
2. Pixel Variance - this should tell us if the model produces images that are too flat or too sharp.
3. Pixel Skewness (3rd moment) - a measure for background-foreground balance
4. Local Patch Variance - computes variance in small 4×4 patches, as a way to measure the local sharpness of the image
5. FID - the standard evaluation metric for generative models that we mentioned in class. Computes the Fréchet distance between the Inception-v3 feature distributions of real and generated images.

Results:

Metric	Real	ODE	SDE
Pixel Mean	-0.762	-0.745	-0.78
Pixel Variance	0.29	0.313	0.275
Pixel Skew	2.359	2.2	2.446
Local patch Variance	0.107	0.108	0.102
FID		274.54	267.03

Overall, the moment-metrics do not reveal some consistent difference between the two methods. The ODE method wins in closer mean and local patch variance, and the SDE wins in Variance and skew.

The FID score, that essentially maps to some deeper-level structure of the generated data, tells a story similar to what we see qualitatively in the previous questions - overall both look very similar to MNIST, with the ODE containing more "artifacts" in the images and also has some more blurry digits.

We do note that the FID values we got are quite high - it might be related to the upsampling we did from 32x32 to 299x299. That said, we still think that these values represent a true difference in the two generated classes

Theoretical Part

Tuesday, 16 June 2026 16:53

1. The equation is: $dX_t = -X_t dt + \sqrt{2} dW_t$ which has equilibrium distribution of $N(0, I)$.
2. Let $x \sim p(x), n \sim N(0,1)$; $u = x + n$ then
   $p_u(u) = \int p(x)\, \phi(u - x) dx$ where ϕ is the gaussian density.
   We know that $u|x = x + n \sim N(0,1) \rightarrow p(u|x) = \frac{1}{\sqrt{2\pi}} e^{-\frac{1}{2}(u-x)^2}$
   With the law ot total probability:
   $p_u(u) = \int p(x)p(u|x)dx = \int p(x) \cdot \frac{1}{\sqrt{2\pi}} e^{-\frac{1}{2}(u-x)^2}\, dx$
   $p(u) = \big(p * \phi\big)(u)$
3. GAN's:
   a. If the latent source distribution z has too small a dimenstion, then the generator $G_\theta(z)$ can only map a low dimensional latent space into the data space. Therefore the generated distribution p_g may be supported only on a low dimensional subset of the true data manifold. As a result, the generator will not be able to represent all variations in the real data distribution, leading to limited diveristy and missing modes.
   b. If the discriminator is much smaller than the generator, then it may not be able to distinguish real samples from generated samples accurately. Since the generator is trained using the discriminator's feedback, a weak discriminator without actually matching the true data distributionm resulting in poor quality samples and bad distribution coverage.
4. We can use the pretrained classifier to guide the smapling process. The unconditional diffusion/score model gives a score direction that pushes the sample towards the general ImageNet distribution, but it does not control the class. For a desired class y , we can evaluate the classifier probability $C(y \mid x)$ on the current sample and use the gradient $\nabla_x \log C\big(\, y \mid x \,\big)$ to push the sample towards images that the classifer assigns to class y. Thus, during each sampling/denoising step, we combine the original generative direction with an additional classifier guidance direction. The diffusion model keeps the image realistic, while the classifier guidance biases it towards the desired class. guided score $= \nabla_x \log p(x) + \lambda \nabla_x \log C\big(\, y \mid x \,\big)$, where λ controls how strongly we force the class.
5. GD and learning rate:
   a. As we have learned GD is: $x^{k+1} = x^k - \eta \cdot \frac{\partial f}{\partial x}$; $y^{k+1} = y^k - \eta \cdot \frac{\partial f}{\partial y}$
   where η is the learning rate. We compute the gradients:
   $\frac{\partial f}{\partial x} = 20(x - 1)$; $\frac{\partial f}{\partial y} = \frac{1}{5}\big(y + 1\big)$ so:

$x^{k+1} = x^k - 20\eta\left(x^k - 1\right)$; $y^{k+1} = y^k - \frac{\eta}{5}\left(y^k + 1\right)$ and for the recommended form:

$x^{k+1} = \left(1 - 20\eta\right)x^k + 20\eta$; $y^{k+1} = \left(1 - \frac{\eta}{5}\right)y^k - \frac{\eta}{5}$

b. For the recurrence form: $z^{k+1} = Az^k + B$ we converge only if $|A| < 1$.
So for x: $A_x = 1 - 20\eta \rightarrow \left|1 - 20\eta\right| < 1 \leftrightarrow -1 < 1 - 20\eta < 1$
$\leftrightarrow 0 < \eta < \frac{1}{10}$
So for y: $A_y = 1 - \frac{\eta}{5} \rightarrow \left|1 - \frac{\eta}{5}\right| < 1 \leftrightarrow 0 < \eta < 10$
So to have both conditions we need $0 < \eta < 0.1$ so the maximal learning threshold is $\eta_{\max} = 0.1$

c. The maximal learning rate is determined by the steepest direction of the quadratic. In our case the x term: $10(x-1)^2$ is much steeper than the y term: $\frac{(y+1)^2}{10}$.
We can also look at the Hessian: $\begin{pmatrix} 20 & 0 \\ 0 & \frac{1}{5} \end{pmatrix}$ and we see that the largest eigen value is $\lambda_{\max} = 20$.
So in terms of the written minimization term, it is the large coefficient 10 in from of the $(x-1)^2$.

d. It is clear to see that the optimal values are $x^* = 1, y^* = -1$ then the expression is at the minima of 0.
We inspect the errors from the optimum:
$e_x^k = x^k - 1$, $e_y^k = y^k + 1 \rightarrow e_x^{k+1} = \left(1 - 20\eta\right)e_x^k$,
$e_y^{k+1} = \left(1 - \frac{\eta}{5}\right)e_y^k$
The variable with update multiplier closer to 1 converges more solwly. Since the y term has very small curvature $\frac{1}{5}$ it takes longer to converge. The value determing this is $\frac{1}{10}$.

6. The Denoising Objective and the Blurring Effect in Diffusion:

a. We consider the objective: $\min_D \mathbb{E}_{x_0,u}\left[\left\|D(u) - x_0\right\|^2\right]$ using the law of total expectation over u is just an average over all possible noisy observations. Therefore, to minimize the total loss, we can minimze the inner expression separartley for each fixed value of u. Fix some noisy observation u. Since $D(u)$ is now just a fixed vector, denote: $a = D(u)$.
Then the problem becomes: $\min_a \mathbb{E}_{x_0|u}\left[\left\|a - x_0\right\|^2\right]$ now we expand the squared norm: $\left\|a - x_0\right\|^2 = \left(a - x_0\right)^T\left(a - x_0\right) =$
$= \|a\|^2 - 2a^T x_o + \left\|x_0\right\|^2$
Taking the conditional expectation given u:
$\mathbb{E}_{x_0|u}\left[\left\|a - x_0\right\|^2\right] = \mathbb{E}_{x_0|u}\left[\|a\|^2 - 2a^T x_o + \left\|x_0\right\|^2\right] =$

$\|a\|^2 - 2a^T\mathbb{E}\left[x_0 \mid u\right] + \mathbb{E}\left[\|x_0\|^2 \mid u\right]$

Now differentiate w.r.t a: $2a - 2\mathbb{E}\left[x_0 \mid u\right] = 0 \leftrightarrow a = \mathbb{E}\left[x_0 \mid u\right]$

So $D^*(u) = \mathbb{E}\left[x_0 \mid u\right]$

b. Let the DDPM forward process be defined by

$q(x_t \mid x_{t-1}) = N(x_t; \sqrt{\alpha_t}x_{t-1}, (1-\alpha_t)I)$, we can also write

$x_t = \sqrt{\alpha_t}x_{t-1} + \sqrt{1-\alpha_t}\epsilon_t \; ; \epsilon_t \sim N(0, I)$

Define $\bar{\alpha}_t = \prod_{s=1}^{t} \alpha_s$ we want to show that

$q(x_t \mid x_0) = N(x_t; \sqrt{\bar{\alpha}_t}x_0, (1-\alpha_t)I)$.

We prove this by induction: for $t = 1$

$x_1 = \sqrt{\alpha_1}x_0 + \sqrt{1-\alpha_1}\epsilon_1$ and since $\bar{\alpha}_1 = \alpha_1$ we get

$x_1 \mid x_0 \sim N\left(\sqrt{\bar{\alpha}_1}x_0, (1-\bar{\alpha}_1)I\right)$

Now we assume it holds for $t-1$

$x_{t-1} = \sqrt{\bar{\alpha}_{t-1}}x_0 + \sqrt{1-\bar{\alpha}_{t-1}}\epsilon'; \epsilon' \sim N(0, I)$

Using the one step forward process

$x_t = \sqrt{\alpha_t}x_{t-1} + \sqrt{1-\alpha_t}\epsilon_t$

We plug in the induction assumption

$x_t = \sqrt{\alpha_t}\left(\sqrt{\bar{\alpha}_{t-1}}x_0 + \sqrt{1-\bar{\alpha}_{t-1}}\epsilon'\right) + \sqrt{1-\alpha_t}\epsilon_t \leftrightarrow$

$x_t = \sqrt{\alpha_t \cdot \bar{\alpha}_{t-1}}x_0 + \sqrt{\alpha_t(1-\bar{\alpha}_{t-1})\epsilon' + \sqrt{1-\alpha_t}\epsilon_t}$

Since $\bar{\alpha}_t = \alpha_t\bar{\alpha}_{t-1}$ the signal term becomes $\sqrt{\bar{\alpha}_t}x_0$

Now for the noise term it is a sum of independent guassians so the covariance is $\alpha_t(1-\bar{\alpha}_{t-1}) + (1-\alpha_t) = \alpha_t\bar{\alpha}_{t-1} = \bar{\alpha}_t$

So we get what we want

c. The blurring effect via convolution:

i. From part b we have $q(x_t \mid x_0) = N\left(x_t; \sqrt{\bar{\alpha}_t}x_0, (1-\bar{\alpha}_t)I\right)$

The marginal distribution of x_t is obtained by integrating over the clean data distribution $p(x_0)$:

$q_t(x_t) = \int q(x_t \mid x_0)p(x_0)dx_0$

Plugging in the gaussian conditional distribution gives

$q_t(x_t) = \int p(x_0)N\left(x_t; \sqrt{\bar{\alpha}_t}x_0, (1-\bar{\alpha}_t)I\right)dx_0$

Or $x_t = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1-\bar{\alpha}_t}\epsilon_t \; ; \epsilon \sim N(0, I)$

So, x_t is obtained by taking the scaled clean data distribution $\sqrt{\bar{\alpha}_t}x_0$ and adding gaussian noise with covariance $(1-\bar{\alpha}_t)I$

So : $q_t = p_{\sqrt{\bar{\alpha}_t}x_0} * N(0, (1-\bar{\alpha}_t)I)$

So the marginal distribution of the noisy data is a convolution between the scaled data distribution and a gaussian noise kernel.

The variance of this gaussian kernel is $\sigma_t^2 = 1 - \bar{\alpha}_t$

Per coordinate, or covarince matrix: $(1-\bar{\alpha}_t)I$

ii. From part a, the optimal denoiser under the ℓ_2 objective is

$D^*(x_t) = \mathbb{E}[x_0 \mid x_t]$, using bayes rule:

$$p(x_0 \mid x_t) = \frac{p(x_0)q(x_t \mid x_0)}{q_t(x_t)}$$

Plugging in from part b:

$$p(x_0 \mid x_t) = \frac{p(x_0)N\left(x_t; \sqrt{\bar{\alpha}_t}x_0, (1-\bar{\alpha}_t)I\right)}{q_t(x_t)}$$

So: $D^*(x_t) = \mathbb{E}[x_0 \mid x_t] = \int x_0 p(x_0 \mid x_t) dx_0 =$

$$= \frac{\int x_0 p(x_0) N\left(x_t; \sqrt{\bar{\alpha}_t}x_0, (1-\bar{\alpha}_t)I\right) dx_0}{\int p(x_0) N\left(x_t; \sqrt{\bar{\alpha}_t}x_0, (1-\bar{\alpha}_t)I\right) dx_0}$$

This expression shows that the optimal ℓ_2 denoiser does not choose one sharp clean image from the posterior distribution. Instead, it returns a weighted average of all clean images x_0 that could have produced the noisy observation x_t

At small time steps, the variance $1 - \bar{\alpha}_t$ is small, so x_t still contains a lot of information about x_0. iInthis case the posterior $p(x_0 \mid x_t)$ is concentrated near the true clean image, so the conditional mean can remain sharp.

At large time steps, $\bar{\alpha}_t$ becomes small and the noise variance $1 - \bar{\alpha}_t$ becomes large. Then x_t contains much less information about the original clean image.

Since $D^*(x_t) = \mathbb{E}[x_0 \mid x_t]$ the denoiser averages over these possible clean images.

In the extreme case where $\bar{\alpha}_t \to 0$ we have $x_t \approx \epsilon$ so x_t is independent of x_0.

Then $p(x_0 \mid x_t) \approx p(x_0)$ so $D^*(x_t) = \mathbb{E}[x_0]$